

RPA Business Case

Freight Invoice Reconciliation Automation

Version	1.0
Status	Final
Prepared by	N M Ziyaf, Supply Chain Business Analyst
Date	30 April 2026
Domain	Freight Forwarding and Supply Chain

This business case follows the Freight Invoice Reconciliation Workflow Gap Analysis BRD and should be read alongside it. The BRD establishes the control gap and the requirements; this document proposes and justifies the automation solution that addresses them.

1. Executive Summary

I prepared this business case to set out the rationale, cost and expected return for automating the freight invoice reconciliation process within our CargoWise One environment. The gap analysis I completed earlier confirmed that carrier invoices are reconciled manually against freight quotes at invoice total level only, with no systematic line-item comparison and no cross-check against the underlying shipment record. The consequence is a financial control weakness rather than a simple efficiency problem, because rate variances embedded in individual charge codes pass through undetected even when the invoice total looks reasonable.

My recommendation is to implement an Automation Anywhere robotic process automation solution that extracts invoice data, performs a three-way match at charge-code line-item level against the freight quote and the shipment record, applies a configurable variance tolerance, and escalates only genuine discrepancies to a human approver. I deliberately rejected the simpler option of automating the existing total-level comparison, because that path would reproduce the current weak control logic at higher speed and would leave the underlying leakage in place. The solution I propose closes the control gap and reduces manual effort at the same time.

The financial case is straightforward. The Year 1 investment is approximately AUD 26,200 and the projected Year 1 benefit is approximately AUD 72,000, giving a net Year 1 benefit of approximately AUD 45,800 and a payback period of around six months. The summary figures are set out below.

Measure	Value
Year 1 total investment	AUD 26,200
Year 1 gross benefit	AUD 72,000
Net Year 1 benefit	AUD 45,800
Payback period	Approximately 6 months
First year return on investment	Approximately 175 percent
Primary outcome	Closure of the three-way matching control gap

2. Problem Statement and Solution Rationale

The reconciliation process today depends on a coordinator opening each carrier invoice, locating the matching freight quote, and comparing the two totals by eye. Invoices arrive in inconsistent formats, some as PDF and some as Excel, which means the coordinator first has to interpret each layout before any comparison can begin. When the totals are close the invoice is approved, and when they differ noticeably it is queried. There is no structured comparison of the individual charge codes that make up the total, and there is no check against the shipment record that would confirm the charges relate to services that were actually performed.

My spot-check sampling showed that only around fifteen percent of invoices received a full comparison against both the quote and the shipment, and that roughly six percent of invoices carried a rate variance that had not been detected. Average processing time was about twenty-two minutes per invoice and the approval cycle averaged 3.1 days. The pattern told me that speed and control were failing together, and that the root cause was the absence of a line-item three-way match rather than a shortage of staff effort.

2.1 Why I Rejected the Simple Automation Approach

The obvious automation, and the one most readily proposed, is to have a bot read each invoice, capture the total, compare it to the quoted total, and auto-approve anything within a rounding margin. I considered this option and rejected it deliberately. The existing control weakness is not that the total comparison is slow, it is that the total comparison is the wrong test. A carrier can overcharge on one charge code and undercharge on another so that the total still reconciles, and an invoice can include charges for services that the shipment record shows were never performed. Automating the total-level check would simply apply the same flawed logic faster, and because automation carries an implicit assumption of correctness, it would most likely reduce the human scrutiny that currently catches some of these cases by chance. The result would be faster processing and weaker control, which is the opposite of what the gap analysis called for.

I therefore set the design principle that automation must strengthen the control, not just accelerate the existing one. That principle is what led me to the three-way matching design rather than the total-level shortcut.

2.2 Why I Chose the Three-Way Matching Design

The solution I propose extracts each invoice into a structured set of charge-code lines, then matches those lines against the corresponding lines on the freight quote and validates them against the shipment record so that every charge is confirmed against both what was agreed and what was actually shipped. Each line is tested against a configurable variance tolerance, so that immaterial differences clear automatically while genuine variances are flagged. Only the flagged exceptions are routed to a human approver, with the matched and mismatched lines presented side by side so the approver can act on the discrepancy directly rather than re-keying the comparison.

This design addresses the actual gap because the test now happens at the level where leakage occurs, which is the individual charge code, and because the shipment record is brought into the

comparison so that phantom charges are caught. It also delivers the efficiency benefit, because the coordinator no longer compares every line on every invoice and instead reviews only the small proportion that genuinely warrants attention. Strengthening the control and reducing the effort are achieved by the same mechanism rather than traded against each other.

3. Cost Benefit Analysis

The figures below are expressed in Australian dollars and cover the first year of operation. Implementation costs are one-off unless noted, and the licence and support line is the recurring component that continues into later years. Benefits are derived from the recovered rate variance that the three-way match makes visible and from the reduction in manual reconciliation effort.

3.1 Implementation Costs

Cost item	Type	Amount (AUD)
RPA design and development	One-off	12,000
Automation Anywhere licence and support, Year 1	Recurring	8,500
Integration and testing with CargoWise One	One-off	3,200
Change management and user training	One-off	1,500
Contingency	One-off	1,000
Total Year 1 investment		26,200

3.2 Projected Benefits

The benefit estimate is intentionally conservative. I applied the recovered variance only to the proportion of spend that the sampling supported, and I valued the effort saving at a fully loaded coordinator rate rather than a base wage.

Benefit item	Basis	Annual value (AUD)
Recovered freight rate variance	Line-item variance now detected and recovered	48,000
Manual effort reduction	Reduced minutes per invoice across annual volume	19,000
Faster approval and reduced late-payment exposure	Shorter approval cycle and fewer query loops	5,000
Total Year 1 benefit		72,000

3.3 Return on Investment

Measure	Value
Year 1 gross benefit	AUD 72,000
Year 1 investment	AUD 26,200
Net Year 1 benefit	AUD 45,800

Measure	Value
First year return on investment	Approximately 175 percent
Payback period	Approximately 6 months
Recurring annual cost from Year 2	AUD 8,500 licence and support

From Year 2 onward the recurring cost falls to the licence and support line of approximately AUD 8,500, while the benefit base continues, so the return improves materially in subsequent years. I have not claimed those later-year gains in the headline payback, which keeps the case grounded in the first year alone.

4. Risk Assessment

I assessed the risks that bear specifically on this solution rather than generic project risks, because the value of the business case depends on these three holding true in operation. Each is recorded below with its likelihood, its impact and the mitigation I propose.

Risk	Likelihood	Impact	Mitigation
Variance tolerance threshold is miscalibrated, so that genuine variances clear automatically or immaterial differences flood the approver	Medium	High	Set the threshold from historical variance data, run it in monitor-only mode during the parallel run, and make it configurable so it can be tuned without redevelopment
Extraction accuracy degrades on carrier invoice formats the bot has not seen before	Medium	Medium	Maintain a format library, route unrecognised layouts to a manual queue rather than guessing, and add new formats through a controlled onboarding step
Quote or shipment data is incomplete or inaccurate, so the three-way match compares against an unreliable baseline	Medium	High	Validate quote and shipment completeness before go-live, fail a match to human review when a reference record is missing, and report data-quality exceptions back to the originating team

5. Implementation Approach

I structured the implementation in five phases so that the control logic is proven before the automation is trusted to run unattended. The parallel run phase is the pivotal control, because it lets the bot and the existing manual process operate on the same invoices at the same time, which confirms that the three-way match and the tolerance threshold behave correctly before any reliance is placed on them.

Phase	Activity	Duration
1. Design and configuration	Confirm charge-code mapping, build the extraction and matching logic, configure the initial tolerance threshold	3 weeks
2. Build and unit testing	Develop the bot, test extraction across known carrier formats, test the matching engine against sample data	4 weeks
3. User acceptance testing	Validate the end-to-end flow against the BRD test scenarios with the reconciliation team	2 weeks
4. Parallel run	Run the bot alongside the manual process on live invoices, compare outcomes, tune the tolerance threshold in monitor-only mode	3 weeks
5. Go-live and stabilisation	Cut over to the automated process, monitor exceptions, onboard remaining carrier formats	2 weeks

5.1 Success Criteria

I will consider the implementation successful when the following conditions are met after the stabilisation period.

1. Every in-scope invoice receives a three-way match at charge-code line-item level, raising match coverage from the baseline of fifteen percent to one hundred percent.
2. Undetected rate variance falls from the baseline of six percent to below one percent of invoices.
3. Average processing time falls from twenty-two minutes to under six minutes per invoice for the exceptions that still require human review.
4. The approval cycle falls from 3.1 days to under one day.
5. The parallel run shows agreement between the bot and the manual process on matched outcomes, with all divergences explained by genuine variances the bot correctly flagged.

6. Recommendation

I recommend that we proceed with the Automation Anywhere three-way matching solution as set out in this business case. The investment of approximately AUD 26,200 returns an estimated AUD 72,000 in the first year and pays back in around six months, and the recurring cost from Year 2 is modest. More importantly, the solution closes the control gap that the gap analysis identified rather than simply processing the existing weak check more quickly, which is the outcome that justifies the work. I recommend that the parallel run phase be treated as a mandatory gate, so that go-live depends on demonstrated agreement between the automated and manual outcomes rather than on schedule pressure.

7. Document Control

Version	Date	Author	Summary of change
0.1	18 April 2026	N M Ziyaf	Initial draft, problem statement and solution rationale
0.2	25 April 2026	N M Ziyaf	Added cost benefit analysis, risk assessment and implementation approach
1.0	30 April 2026	N M Ziyaf	Finalised recommendation and figures for review